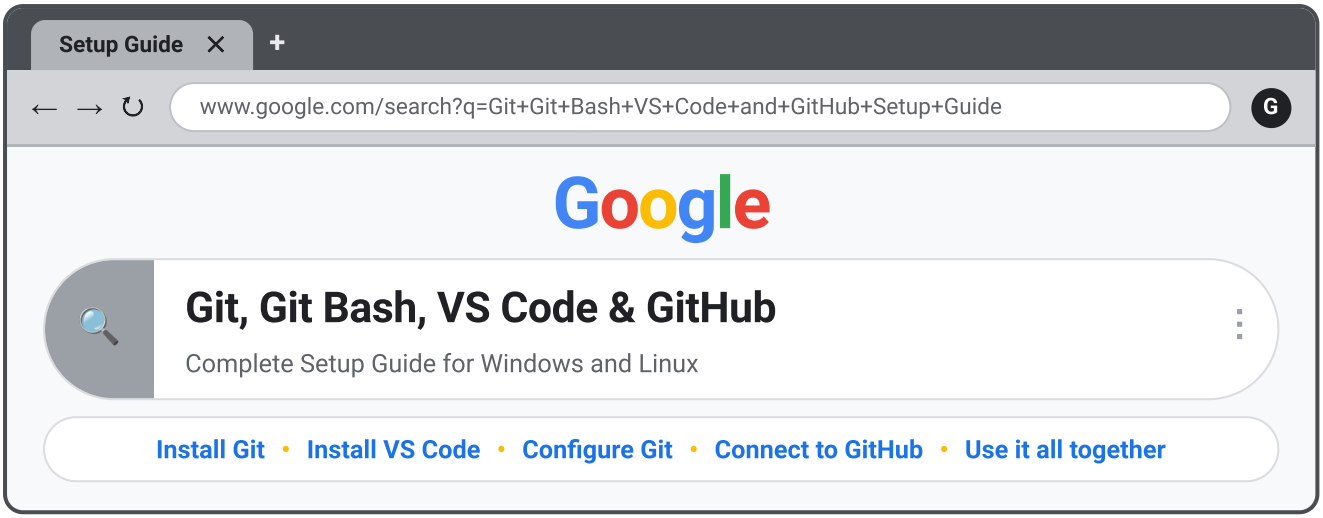


Part	Topic	Applies to
1	Prerequisites and overview	Both
2	Install Git, Git Bash and VS Code on Windows	Windows
3	Install Git and VS Code on Linux	Linux
4	Configure Git (name, email, defaults)	Both
5	Create a GitHub account	Both
6	Connect Git to GitHub (SSH key or HTTPS token)	Both
7	Connect VS Code to Git and GitHub	Both
8	Your first repository: clone, commit, push	Both
9	Command cheat sheet	Both
10	Troubleshooting	Both
11	All links in one place	Both

How to use this guide. Follow the steps in order. Every command is shown in a grey box with a blue label telling you where to run it (Git Bash, a Linux terminal, VS Code terminal, or a web page). Replace anything written in CAPITAL letters (e.g. `YOUR_EMAIL`) with your own details.

Part 1: Prerequisites and Overview

What each tool does

Tool	Purpose
Git	Version-control system that tracks changes to your files on your computer.
Git Bash (Windows only)	A Bash terminal that comes bundled with Git for Windows. It lets you use Linux-style commands on Windows. On Linux you do not need it: the normal terminal already runs Bash.
VS Code	Free code editor from Microsoft with built-in Git support (Source Control panel).
GitHub	Website that hosts your Git repositories online so you can back them up and collaborate.

What you need before starting

- A computer running Windows 10/11 (64-bit) or a modern Linux distribution (Ubuntu, Debian, Fedora, Arch, etc.).
- Internet connection and permission to install software (administrator on Windows, `sudo` on Linux).
- An email address for your GitHub account.

> **Order of work:** Install Git → Install VS Code → Configure Git → Create GitHub account → Connect Git to GitHub → Connect VS Code. Do not skip the configuration step; Git will refuse to commit without a name and email.

Part 2: Windows Installation

Step 2.1: Download and install Git for Windows (includes Git Bash)

1. Open https://git-scm.com/download/win in your browser. The download of the 64-bit installer usually starts automatically; if not, click **Click here to download** / **64-bit Git for Windows Setup**.
2. Double-click the downloaded `Git-x.xx.x-64-bit.exe` file and click **Yes** on the admin prompt.
3. Click **Next** through the screens. Choose the following options where they appear:

Installer screen	Recommended choice
Select Components	Keep defaults. Make sure **Git Bash Here** (Windows Explorer integration) is ticked.
Choosing the default editor	Select **Use Visual Studio Code as Git's default editor** (if VS Code is not listed yet, pick any editor; you will set it properly in Part 4).
Initial branch name	Select **Override the default branch name** and type `main`.

Installer screen	Recommended choice
Adjusting your PATH	Select **Git from the command line and also from 3rd-party software** (recommended).
SSH executable	**Use bundled OpenSSH**.
HTTPS transport backend	**Use the OpenSSL library** (or **Windows Secure Channel** on a company network).
Line ending conversions	**Checkout Windows-style, commit Unix-style line endings** (default).
Terminal emulator for Git Bash	**Use MinTTY** (default).
Default behavior of git pull	**Fast-forward or merge** (default).
Credential helper	**Git Credential Manager** (default; it handles GitHub sign-in for HTTPS).
Extra options	Keep **Enable file system caching** ticked.

4. Click **Install**, wait for it to finish, then click **Finish**.

Step 2.2: Verify Git and open Git Bash

Press the Windows key, type **Git Bash** and press Enter. A black/white terminal window opens. Type the command below:

RUN IN: Git Bash

```
git --version
# Expected output example: git version 2.4x.x.windows.1
```

You can also open Git Bash inside any folder: right-click in the folder in File Explorer and choose **Show more options > Git Bash Here** (Windows 11) or **Git Bash Here** (Windows 10).

Step 2.3: Download and install VS Code

1. Go to https://code.visualstudio.com/download and click the **Windows** download (User Installer, 64-bit is fine for most people).
2. Run `VSCodeUserSetup-x64-x.xx.x.exe`. Accept the agreement and click **Next**.
3. On **Select Additional Tasks**, tick all of these:
   - **Add "Open with Code" action to Windows Explorer file context menu**
   - **Add "Open with Code" action to Windows Explorer directory context menu**
   - **Register Code as an editor for supported file types**
   - **Add to PATH** (usually ticked by default; this makes the `code` command work)
4. Click **Install**, then **Finish**.

Step 2.4: Verify VS Code from Git Bash

Close and reopen Git Bash (so it picks up the new PATH), then run:

RUN IN: Git Bash

```
code --version
```

A version number such as `1.9x.x` should appear.

Step 2.5: Make Git Bash the default terminal inside VS Code

1. Open VS Code. Press **Ctrl+Shift+P** to open the Command Palette.
2. Type `Terminal: Select Default Profile` and press **Enter**.
3. Choose **Git Bash** from the list.
4. Open a new terminal with **Ctrl+`** (backtick key) or the menu **Terminal > New Terminal**. The prompt should now look like Git Bash (you will see `MINGW64` in the prompt).

> **Alternative (settings file):** press **Ctrl+Shift+P**, run `Preferences: Open User Settings (JSON)` and add the line `"terminal.integrated.defaultProfile.windows": "Git Bash"` inside the braces.

Part 3: Linux Installation

On Linux there is no separate "Git Bash". Your normal terminal (GNOME Terminal, Konsole, etc.) already runs Bash, so you use it the same way. Open it with **Ctrl+Alt+T** (Ubuntu/Debian) or search for "Terminal" in your applications menu.

Step 3.1: Install Git

Pick the commands for your distribution. Official reference: https://git-scm.com/download/linux

Ubuntu / Debian / Linux Mint:

RUN IN: Linux Terminal

```
sudo apt update
sudo apt install git -y
```

Fedora / RHEL / CentOS Stream / Rocky / Alma:

RUN IN: Linux Terminal

```
sudo dnf install git -y
```

Arch / Manjaro:

RUN IN: Linux Terminal

```
sudo pacman -S git
```

openSUSE:

RUN IN: Linux Terminal

```
sudo zypper install git
```

Step 3.2: Verify Git

RUN IN: Linux Terminal

```
git --version
```

Step 3.3: Install VS Code

Official download page: https://code.visualstudio.com/download. Choose one of the methods below.

Method A: Ubuntu / Debian using the .deb package

1. Download the **.deb (x64)** file from the page above (it usually saves in `~/Downloads`). Then run:

RUN IN: Linux Terminal

```
cd ~/Downloads
sudo apt install ./code_*.deb
```

Method B: Fedora / RHEL / CentOS using Microsoft's repository

RUN IN: Linux Terminal

```
sudo rpm --import https://packages.microsoft.com/keys/microsoft.asc
sudo tee /etc/yum.repos.d/vscode.repo > /dev/null << 'EOT'
[code]
name=Visual Studio Code
baseurl=https://packages.microsoft.com/yumrepos/vscode
enabled=1
gpgcheck=1
gpgkey=https://packages.microsoft.com/keys/microsoft.asc
EOT
sudo dnf check-update
sudo dnf install code -y
```

Method C: Any distribution with Snap

RUN IN: Linux Terminal

```
sudo snap install code --classic
```

Method D: Arch Linux (community build; AUR helper required, e.g. yay)

RUN IN: Linux Terminal

```
yay -S visual-studio-code-bin
```

Step 3.4: Verify VS Code

RUN IN: Linux Terminal

```
code --version
```

To launch VS Code from the terminal in the current folder later, you will use `code .`

On Linux, VS Code uses your system's default terminal shell (usually Bash), so no extra terminal setup is needed. Open its terminal with **Ctrl+`**.

Part 4: Configure Git (Windows and Linux)

Run these commands once on each computer. Use Git Bash on Windows and the Linux Terminal on Linux. Use the same email that you will use (or add) in your GitHub account.

Step 4.1: Set your identity

RUN IN: Git Bash (Windows) / Linux Terminal

```
git config --global user.name "YOUR_NAME"
git config --global user.email "YOUR_EMAIL@example.com"
```

Step 4.2: Set the default branch name and editor

RUN IN: Git Bash (Windows) / Linux Terminal

```
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
```

Step 4.3: Line-ending setting (choose the one for your OS)

Windows (converts to Windows line endings on checkout, Unix on commit):

RUN IN: Git Bash

```
git config --global core.autocrlf true
```

Linux (keeps Unix line endings):

RUN IN: Linux Terminal

```
git config --global core.autocrlf input
```

Step 4.4: Verify your configuration

RUN IN: Git Bash (Windows) / Linux Terminal

```
git config --list
# or check single values:
git config user.name
git config user.email
```

Part 5: Create a GitHub Account

1. Go to https://github.com/signup and enter your email, a password and a username.
2. Verify the puzzle and the code sent to your email.
3. Sign in at https://github.com/login. (Optional but recommended: enable two-factor authentication at https://github.com/settings/security.)
4. **Privacy tip:** if you do not want your real email in public commits, open https://github.com/settings/emails, tick **Keep my email addresses private** and copy the address shown as `ID+USERNAME@users.noreply.github.com`. Use that value in Step 4.1 instead of your real email.

Part 6: Connect Git to GitHub

Git needs a way to prove to GitHub that you are you. Choose one method. GitHub no longer accepts your account password for Git operations over HTTPS.

Method	Best for	Notes
A. SSH key (recommended)	Daily use on your own computer	Set up once, no repeated logins.
B. HTTPS + sign-in / token	Corporate networks that block SSH (port 22)	On Windows, Git Credential Manager opens a browser sign-in automatically.
C. GitHub CLI (gh)	Quick login with one command	Optional; works on both OS.

Method A: SSH key

Step A1. Generate the key. Press `Enter` to accept the default file location. A passphrase is optional but recommended.

RUN IN: Git Bash (Windows) / Linux Terminal

```
ssh-keygen -t ed25519 -C "YOUR_EMAIL@example.com"
```

Step A2. Start the SSH agent and add your key.

RUN IN: Git Bash (Windows) / Linux Terminal

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Step A3. Copy the public key (the file ending in `.pub`; never share the one without `.pub`).

Windows:

RUN IN: Git Bash

```
clip < ~/.ssh/id_ed25519.pub
```

Linux (select and copy the printed text, or use xclip if installed):

RUN IN: Linux Terminal

```
cat ~/.ssh/id_ed25519.pub
# optional, if xclip is installed:
xclip -selection clipboard < ~/.ssh/id_ed25519.pub
```

Step A4. Add the key to GitHub. Open https://github.com/settings/keys and click **New SSH key**. Enter a title (e.g. "My Laptop"), keep **Key type: Authentication Key**, paste the key into the box and click **Add SSH key**.

Step A5. Test the connection. Type `yes` if asked about the host fingerprint.

RUN IN: Git Bash (Windows) / Linux Terminal

```
ssh -T git@github.com
```

Success looks like: `Hi USERNAME! You've successfully authenticated, but GitHub does not provide shell access.` (This is normal.)

> Remote URLs for the SSH method look like `git@github.com:USERNAME/REPO.git`. Always choose the **SSH** tab when copying the clone URL from GitHub.

Method B: HTTPS with sign-in or Personal Access Token

Windows: No extra setup. The first time you run `git push` or `git clone` on a private repo using an `https://` URL, Git Credential Manager opens a window. Choose **Sign in with your browser** and approve it on GitHub. The login is then saved.

Linux: Create a token and tell Git where to store it.

1. Open https://github.com/settings/tokens > **Generate new token (classic)** (or a fine-grained token). Give it a name, an expiry date, and tick the `repo` scope. Click **Generate token** and copy it immediately (it is shown only once).
2. Tell Git to remember credentials. The `cache` option keeps them in memory for 1 hour; `store` saves them in plain text (less secure).

RUN IN: Linux Terminal

```
git config --global credential.helper 'cache --timeout=3600'
# or (plain-text file, less secure):
git config --global credential.helper store
```

3. On your next `git push` over HTTPS, enter your GitHub username and paste the token as the password.

Method C (optional): GitHub CLI

Download from https://cli.github.com (Windows installer) or install on Linux using the instructions at https://github.com/cli/cli/blob/trunk/docs/install_linux.md. Then log in:

RUN IN: Git Bash (Windows) / Linux Terminal

```
gh auth login
# Choose: GitHub.com > SSH or HTTPS > Login with a web browser
```

Part 7: Connect VS Code to Git and GitHub

Step 7.1: Confirm VS Code finds Git

VS Code detects Git automatically if it is installed and on your PATH. To check, open VS Code and click the **Source Control** icon in the left bar (or press `Ctrl+Shift+G`). If you see a message saying Git was not found, restart VS Code. If it persists, set the path manually:

- Press `Ctrl+,` to open Settings and search for `git.path`.
- Windows value example: `C:\Program Files\Git\cmd\git.exe`
- Linux value example: `/usr/bin/git` (find it with `which git`).

Step 7.2: Open a project folder

Use **File > Open Folder...** or run the following from Git Bash / Linux Terminal inside your project folder:

RUN IN: Git Bash (Windows) / Linux Terminal

```
cd ~/projects/my-project
code .
```

When asked **Do you trust the authors of the files in this folder?** choose **Yes, I trust the authors** only for folders you created or trust.

Step 7.3: Sign in to GitHub from VS Code

- Click the **Accounts** icon (person silhouette) at the bottom-left of the VS Code window.
- Choose **Sign in with GitHub to use GitHub Pull Requests...** (or **Backup and Sync Settings** if you want settings sync).
- Your browser opens at `github.com`. Click **Authorize Visual-Studio-Code**, then allow it to open VS Code.
- Back in VS Code, your GitHub username now appears under the Accounts icon.

Step 7.4: Install helpful extensions (optional)

Press `Ctrl+Shift+X` to open Extensions, search and click **Install**:

- **GitHub Pull Requests** (publisher: GitHub) — manage pull requests and issues inside VS Code.
- **GitLens** (publisher: GitKraken) — shows who changed each line and rich history.
- **Git Graph** — visual commit history (optional).

Step 7.5: Using the Source Control panel

Action in VS Code	What it does	Equivalent command
Click **+** next to a file	Stage the change	`git add FILE`
Type a message and click **Commit**	Save a snapshot locally	`git commit -m "message"`
Click **Sync Changes** / **Publish Branch**	Push/pull with GitHub	`git push` / `git pull`
Bottom-left branch name	Switch or create branches	`git switch BRANCH`

Part 8: Your First Repository (End-to-End Test)

Option 1: Start from GitHub and clone (easiest)

Step 1. Go to https://github.com/new, enter a repository name (e.g. `hello-git`), tick **Add a README file**, and click **Create repository**.

Step 2. Click the green **Code** button, choose the **SSH** tab (or **HTTPS** if you used Method B) and copy the URL.

Step 3. Clone it and open it in VS Code:

RUN IN: Git Bash (Windows) / Linux Terminal

```
cd ~
mkdir -p projects && cd projects
git clone git@github.com:YOUR_USERNAME/hello-git.git
cd hello-git
code .
```

Step 4. Edit `README.md` in VS Code and save it. Then, in the VS Code terminal (`Ctrl+``) or Git Bash:

RUN IN: VS Code Terminal / Git Bash / Linux Terminal (inside the project folder)

```
git status
git add .
git commit -m "Update README"
git push
```

Refresh the repository page on GitHub. You should see your change and the commit message.

Option 2: Start locally and push to a new GitHub repo

First create an empty repository on https://github.com/new (do not tick README), then run:

RUN IN: Git Bash (Windows) / Linux Terminal

```
mkdir my-project && cd my-project
git init
echo "# My Project" > README.md
git add README.md
git commit -m "First commit"
git branch -M main
git remote add origin git@github.com:YOUR_USERNAME/my-project.git
git push -u origin main
```

> **Using HTTPS instead of SSH?** Replace the URL with `https://github.com/YOUR_USERNAME/REPO.git`.

Part 9: Command Cheat Sheet

Command	Purpose
`git status`	Show changed, staged and untracked files
`git add FILE` / `git add .`	Stage one file / all changes
`git commit -m "msg"`	Save staged changes with a message
`git push` / `git push -u origin main`	Upload commits to GitHub (`-u` sets upstream the first time)
`git pull`	Download and merge new changes from GitHub
`git clone URL`	Copy a GitHub repo to your computer
`git log --oneline`	Short history of commits
`git diff`	Show unstaged changes
`git branch` / `git switch -c NAME`	List branches / create and switch to a new branch
`git remote -v`	Show linked GitHub URLs
`git restore FILE`	Discard uncommitted changes to a file

Part 10: Troubleshooting

Problem	Cause	Fix
'git' is not recognized (Windows)	PATH not updated or terminal opened before install	Close all terminals and reopen. If still failing, re-run the Git installer and select **Git from the command line and also from 3rd-party software**.
'code' is not recognized	VS Code was installed without Add to PATH	Re-run the VS Code installer with **Add to PATH** ticked, or in VS Code press **Ctrl+Shift+P** and run **Shell Command: Install 'code' command in PATH** (Linux/macOS). Reopen the terminal.
Permission denied (publickey)	GitHub does not have your public key, or the agent does not have the private key	Redo steps A2–A4, then run `ssh -T git@github.com`. Check with `ssh-add -l` that your key is loaded.
Authentication failed / password not supported (HTTPS)	GitHub does not accept account passwords over HTTPS	Use a Personal Access Token as the password, or sign in through Git Credential Manager, or switch to SSH.

Problem	Cause	Fix
Author identity unknown / Please tell me who you are	`user.name` / `user.email` not set	Run the commands in Step 4.1.
Host key verification failed	Fingerprint not accepted	Run `ssh -T git@github.com` and answer `yes`. Compare with fingerprints at GitHub docs.
'LF will be replaced by CRLF' warning (Windows)	Line-ending conversion notice	Harmless. Keep `core.autocrlf true` (Step 4.3).
Rejected push: remote contains work you do not have	Someone (or GitHub's README) changed the remote first	Run `git pull` to merge, then `git push` again.
Terminal in VS Code is not Git Bash (Windows)	Default profile not set	Repeat Step 2.5.

Part 11: All Links in One Place

Purpose	Link
Git for Windows (includes Git Bash)	https://git-scm.com/download/win
Git for Linux install instructions	https://git-scm.com/download/linux
Git documentation and book	https://git-scm.com/doc
VS Code download (Windows/Linux)	https://code.visualstudio.com/download
VS Code Git documentation	https://code.visualstudio.com/docs/sourcecontrol/overview
Create a GitHub account	https://github.com/signup
GitHub SSH keys settings	https://github.com/settings/keys
GitHub personal access tokens	https://github.com/settings/tokens
GitHub email privacy settings	https://github.com/settings/emails
Create a new GitHub repository	https://github.com/new
GitHub CLI	https://cli.github.com
GitHub official docs (connecting to GitHub)	https://docs.github.com/en/authentication

> **Note:** Installer screens and menu names may change slightly with new versions. If a screen looks different, choose the option closest to the one described, or refer to the official links above.